

FSD - Lecture 4 Notes

DOUBT LINK :

<https://docs.google.com/forms/d/e/1FAIpQLSdhCcp7DKozrWqy-3V1mTvn5WbXgZaxdhxceV-Z5KH29ynJg/viewform?usp=publish-editor>

Synchronous Programming (SYNC)

Definition

Synchronous programming means:

- Tasks are executed one after another.
- Each task must finish before the next one starts.

JS waits. No shortcuts.

♦ Example

```
console.log("1");  
console.log("2");  
console.log("3");
```

Output

```
1  
2  
3
```

Problem with Sync

If a task is slow (API, timer, file read):

- Everything freezes
- Bad user experience

2 Asynchronous Programming (ASYNC)

✓ Definition

Asynchronous programming means:

- Long tasks run in the background,
- JS continues executing other code.

JS doesn't wait like an idiot.

♦ Example

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Middle");  
}, 2000);
```

```
console.log("End");
```

Output

```
Start  
End  
Middle
```

Why? Because `setTimeout` is async.

🌐 Real-world Analogy

Restaurant order

- You order food
- Chef cooks (background)
- You chat / scroll phone
- Food arrives later

3 JavaScript & Asynchronous Nature

Important truth:

JavaScript is SINGLE-THREADED but NON-BLOCKING

Means:

- One thing at a time
- But async tasks don't block execution

JS uses:

- Call Stack
- Web APIs
- Callback Queue
- Event Loop

Promises (Better way)

Definition

A Promise is:

An object that represents a value
that will be available now, later, or never

States:

- `pending`
- `fulfilled`
- `rejected`

♦ Example

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {
```

```
    resolve("Data received");  
  }, 2000);  
});
```

```
promise.then((result) => {  
  console.log(result);  
});
```

Output:

Data received

Real-world Analogy

Amazon order

- You place order → *pending*
 - Delivered → *fulfilled*
 - Cancelled → *rejected*
-

Why **.then(data => ...)**?

Because:

```
resolve("hello");
```

means

👉 "Send **"hello"** to whoever is waiting"

So this:

```
.then(data => console.log(data))
```

receives **"hello"**.

async keyword

✓ Definition

`async` makes a function:

Always return a Promise

Even if you return a normal value.

♦ Example

```
async function test() {  
  return "Hello";  
}
```

```
test().then(res => console.log(res));
```

Output

Hello

Why? Because `async` auto-wraps in Promise.

await keyword

✓ Definition

`await` means:

“Pause execution inside `async` function
until the Promise is resolved”

⚠ Only works inside `async`

♦ Example

```
function getData() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve("Hello");  
    }, 2000);  
  });  
}
```

```
async function showData() {  
  const data = await getData();  
  console.log(data);  
}
```

```
showData();
```

Output (after 2 sec)

Hello

Real-world Analogy

`await` =

“Wait here, but don’t block the whole world”

Like waiting for food at your table, not standing in the kitchen.

